Laporan Tugas Besar 1
IF3270 Pembelajaran Mesin

Disusun oleh:

Kelompok Feeding Frenzy Neural Network

Wisa Ahmaduta DInutama (18223003)

Muhammad Faiz Alfikrona (18223009)

Ni Made Sekar Jelita (18223101)

Kelas K1

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

Laporan Tugas Besar 1 ..... **1**
IF3270 Pembelajaran Mesin **1**
Daftar Isi **2**
Deskripsi Persoalan **3**
Pembahasan **4**
1. Penjelasan Implementasi 4
2. Hasil Pengujian 14
Kesimpulan dan Saran **19**
1. Kesimpulan 19
2. Saran 19
Pembagian Tugas **20**
Referensi **21**
SURAT PERNYATAAN PENGGUNAAN AI **22**

Deskripsi Persoalan

Tugas Besar 1 Pembelajaran Mesin berfokus pada implementasi Feedforward Neural Network (FFNN) dari awal tanpa bergantung pada library yang siap pakai menggunakan operasi-operasi yang dipelajari saat perkuliahan. Mahasiswa diharapkan dapat memahami cara kerja jaringan saraf tiruan dari inisialisasi parameter sampai pembaruan bobot (*backpropagation*).

Implementasi diharapkan mampu mengakomodasi variasi kedalaman layer dan lebar tiap layer, mendukung banyak metode inisialisasi bobot, mekanisme *forward propagation*, *backward propagation*, fungsi aktivasi, dan fungsi loss.

Setelah jaringan saraf dibangun, model diharapkan dapat mempelajari dan memprediksi persoalan mengenai Global Student Placement & Salary Dataset, di mana dataset ini berisi performa akademik dan nonakademik mahasiswa seperti *cumulative* GPA, *backlog*, internship, serta tes kemampuan kuantitatif dan komunikasi. Targetnya adalah memprediksi prospek karier mahasiswa setelah lulus dengan target variabel kategorikal pada kolom `placement_status`.

Tahapan paling penting adalah pengujian model terhadap dataset tersebut dengan memvariasikan *hyperparameter* seperti lebar dan kedalaman jaringan, penggunaan fungsi aktivasi, hingga penyesuaian *learning rate*. Terakhir, model akan dibandingkan dengan implementasi Multi Layer Perceptron dari scikit-learn.

Pembahasan

1. Penjelasan Implementasi

a. Kelas, Atribut, Method

Tabel 1.1 Kelas, Atribut, dan Method pada activations.py

Kelas	Deskripsi	Atribut	Method
Linear	Implementasi fungsi identitas dengan output sama dengan input	• self.input: menyimpan nilai input terakhir untuk keperluan backward pass	• forward(x): Mengembalikan x tanpa perubahan • backward(): Mengembalikan gradien bernilai 1 dengan shape sesuai input
ReLu	Implementasi Rectified Linear Unit dengan output nilai negatif	• self.input: menyimpan nilai input untuk menentukan posisi gradien 0 atau 1	• forward(x): Menghitung max(0, x) • backward(): Mengembalikan 1 jika x>0 dan 0 jika x <= 0
Sigmoid	Fungsi aktivasi yang memetakan input ke rentang (0, 1)	• self.input: menyimpan hasil perhitungan forward agar efisien saat menurunkan gradien	• forward(x): Menghitung $\frac{1}{1+e^{-x}}$ • backward(): Menghitung turunan menggunakan rumus $\sigma(x)(1 - \sigma(x))$
Tanh	Fungsi Hyperbolic Tangent yang memetakan input ke rentang (-1, 1)	• self.input: Menyimpan hasil perhitungan tanh terakhir	• forward(x): Menghitung $\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$ • backward(): Menghitung turunan menggunakan

			rumus $1 - tanh^2(x)$
Softmax	Digunakan pada layer output untuk menghasilkan distribusi probabilitas	• self.input: Menyimpan probabilitas output untuk setiap kelas dalam batch	• forward(x): Menghitung eksponensial ternormalisasi dengan stabil secara numerik • backward(): Menghitung matriks Jacobian untuk setiap sampel dalam batch
Helper Function			
Nama Fungsi	**Deskripsi**	**Parameter**	**Return**
get_activation	Fungsi pembantu untuk memetakan nama string ke instance kelas aktivasi yang sesuai.	• name (str): Nama fungsi aktivasi (misal: "relu", "sigmoid")	• Mengembalikan objek (instance) dari kelas aktivasi yang diminta

Tabel 1.2 Kelas, Atribut, dan Method pada initializers.py

Kelas	Deskripsi	Atribut	Method
ZeroInitializer	Menginisialisasi semua bobot dengan nilai nol	-	• initialize(shape): Mengembalikan array berisi nol dengan dimensi sesuai parameter shape
UniformInitializer	Menghasilkan bobot acak yang terdistribusi secara merata dalam rentang tertentu	• lower: batas bawah nilai acak • upper: batas atas nilai acak • Seed: parameter untuk me-*lock random number generator*	• initialize(shape): Menghasilkan matriks/vektor acak dengan distribusi uniform berdasarkan rentang yang

			ditentukan
NormalInitializer	Menghasilkan bobot acak dengan distribusi normal (Gaussian)	• mean: Rata-rata distribusi • std: Standar deviasi (akar dari variansi) • seed: Parameter untuk reproduksibilitas	• initialize(shape): Menghasilkan matriks/vektor acak dengan distribusi normal.
XavierInitializer	Inisialisasi yang menjaga variansi sinyal tetap konstan antar layer, ideal untuk aktivasi Sigmoid/Tanh	• seed: Parameter untuk reproduksibilitas	• initialize(shape): Menghitung batas (limit) berdasarkan n_in dan n_out, lalu melakukan sampling uniform
HeInitializer	Inisialisasi yang dirancang khusus untuk fungsi aktivasi ReLU	• seed: Parameter untuk reproduksibilitas	• initialize(shape): Menghitung batas berdasarkan jumlah neuron input (n_in) untuk melakukan sampling
Helper Function			
Nama Fungsi	**Deskripsi**	**Parameter**	**Return**
get_initializer	Fungsi pembantu untuk memilih dan melakukan konfigurasi objek inisialisasi berdasarkan input string	• name (str): Jenis inisialisasi (misal: "uniform", "xavier"). • Argumen tambahan untuk parameter spesifik (seperti seed atau variance).	• Instance dari kelas inisialisasi yang diminta.

Tabel 1.3 Kelas, Atribut, dan Method pada layers.py

Kelas	Deskripsi	Atribut	Method
DenseL	Kelas ini	• n_in: Integer yang	• __init__:

ayer	mengimplementasikan lapisan Dense (lapisan terhubung penuh). Kelas ini menyimpan parameter bobot (W) dan bias (b), serta menangani proses *forward* dan aliran *backward*	menyatakan jumlah neuron input yang masuk ke layer • n_out: Integer yang menyatakan jumlah neuron output yang ada pada layer • activation: Objek fungsi aktivasi yang akan diterapkan pada hasil operasi linear • regularizer: Objek regulerisasi yang digunakan untuk menghitung penalti pada bobot agar dapat mencegah overfitting • W: Matriks bobot berukuran (n_in, n_out) yang diinisialisasi menggunakan strategi tertentu. • B: Vektor bias berukuran (1, n_out) yang diinisialisasi dengan nilai nol • dW: Matriks yang menyimpan akumulasi gradien terhadap bobot W hasil dari proses backward • db: Vektor yang menyimpan akumulasi gradien terhadap bias b hasil dari proses backward • Input_cache: Menyimpan data input x dari proses forward untuk digunakan kembali saat menghitung gradien di proses backward	Constructor untuk menyiapkan dimensi layer, menginisialisasi bobot dan bias, serta menetapkan fungsi aktivasi dan regulerisasi yang digunakan • forward(x): Melakukan operasi z=xW+b dan mengembalikan nilai $a = activation(z)$ untuk mendukung pemrosesan data dalam bentuk batch • backward(upstream_grad): Menghitung gradien lokal terhadap W dan b berdasarkan upstream_grad (gradien dari layer berikutnya) menggunakan *chain rule*. Method ini juga menghitung dx untuk diteruskan ke layer sebelumnya

		• Z_cache: Menyimpan hasil operasi linear $(xW + b)$ sebelum diaplikasikan fungsi aktivasi • A_cache: Menyimpan hasil akhir output layer setelah fungsi aktivasi $(a = f(z)$	

Tabel 1.4 Kelas, Atribut, dan Method pada losses.py

Kelas	Deskripsi	Atribut	Method
MSE	Menghitung rata-rata kuadrat selisih antara target dan prediksi	-	• forward(y_true, y_pred): Menghitung nilai $\frac{1}{n}\Sigma(y_i - \widehat{y}_i)^2$ • backward(y_true, y_pred): Menghitung gradien $\frac{-2}{n}(y_i - \widehat{y}_i)$
BinaryCrossEntropy	Digunakan untuk klasifikasi biner. Mengukur performa model yang outputnya berupa probabilitas antara 0 dan 1	-	• forward(y_true, y_pred): Menghitung log-loss biner dengan clipping untuk stabilitas • backward(y_true, y_pred): Menghitung turunan BCE terhadap prediksi.
CategoricalCrossEntropy	Digunakan untuk klasifikasi *multi-class*. Membandingkan distribusi probabilitas prediksi dengan label *one-hot encoded*	-	• forward(y_true, y_pred): Menghitung total log-loss untuk semua kelas • backward(y_true,

			y_pred): Menghitung gradien
HeInitializer	Inisialisasi yang dirancang khusus untuk fungsi aktivasi ReLU	• seed: Parameter untuk reproduksibilitas	• initialize(shape): Menghitung batas berdasarkan jumlah neuron input (n_in) untuk melakukan sampling $\frac{-1}{n}$ $(\frac{y_{ij}}{\hat{y}_{ij}})$
Helper Function			
Nama Fungsi	**Deskripsi**	**Parameter**	**Return**
get_loss	Fungsi pembantu untuk menginstansiasi objek loss berdasarkan nama string yang diberikan.	• name (str): Nama fungsi loss (misal: "mse", "bce", "cce")	• Mengembalikan instance dari kelas kerugian yang diminta.

Tabel 1.5 Kelas, Atribut, dan Method pada layers.py

Kelas	Deskripsi	Atribut	Method
FFNN	Kelas utama yang mengimplementasikan arsitektur Feedforward Neural Network. Berfungsi mengintegrasikan lapisan, menghitung loss, melakukan optimasi bobot melalui backpropagation, serta menyediakan antarmuka untuk menyimpan/mem	• layers: List yang menampung objek-objek DenseLayer yang membentuk arsitektur jaringan saraf • loss_fn: Objek fungsi loss yang digunakan untuk menghitung selisih antara prediksi dan target (diperoleh melalui get_loss) • history: Dictionary yang menyimpan catatan training loss dan validation loss di	• __init__: Menginisialisasi arsitektur jaringan berdasarkan jumlah neuron, jenis aktivasi, inisialisasi bobot, dan strategi regulerisasi yang diberikan • forward(x): Melakukan *forward propagation* dengan mengalirkan input X melalui setiap layer secara berurutan hingga menghasilkan output final • predict(x): Melakukan inferensi pada data

	uat model	setiap epoch untuk keperluan analisis grafik • regularizer: Objek regulerisasi yang digunakan untuk menghitung penalti pada bobot agar dapat mencegah overfitting • W: Matriks bobot berukuran (n_in, n_out) yang diinisialisasi menggunakan strategi tertentu. • B: Vektor bias berukuran (1, n_out) yang diinisialisasi dengan nilai nol • dW: Matriks yang menyimpan akumulasi gradien terhadap bobot W hasil dari proses backward • db: Vektor yang menyimpan akumulasi gradien terhadap bias b hasil dari proses backward • Input_cache: Menyimpan data input x dari proses forward untuk digunakan kembali saat menghitung gradien di proses backward • Z_cache: Menyimpan hasil operasi linear $(xW + b)$ sebelum	input. Method ini secara otomatis mengubah output probabilitas menjadi label kelas • backward(y_true, y_pred): Menghitung gradien secara berurutan dari layer terakhir kembali ke layer pertama (backpropagation) menggunakan hasil dari fungsi loss • update_weights(lr): Memperbarui parameter W dan b pada setiap layer berdasarkan gradien yang telah dihitung dikalikan dengan learning rate • compute_loss(X, y): Menghitung total error model pada data tertentu, termasuk penambahan pinalti dari komponen regulerisasi (L1/L2) • fit(self, X_train: np.ndarray, y_train: np.ndarray,batch_size : int, learning_rate: float, epochs: int, verbose: int = 1, validation_data=None): Melakukan proses pelatihan model menggunakan algoritma Mini-batch Gradient Descent dengan pengacakan data (shuffling) di

		diaplikasikan fungsi aktivasi • A_cache: Menyimpan hasil akhir output layer setelah fungsi aktivasi ($a = f(z)$)	setiap epoch • save(filepath): Menyimpan bobot, bias, dan riwayat pelatihan model ke dalam file menggunakan format pickle • load(filepath): Memuat kembali parameter model dan riwayat pelatihan yang telah disimpan sebelumnya • plot_weight_distirbution: Menghasilkan histogram yang menunjukkan sebaran nilai bobot pada layer tertentu sesuai indeks yang dipilih • plot_gradient_distribution: Menghasilkan histogram yang menunjukkan sebaran nilai gradien pada layer tertentu untuk memantau stabilitas pelatihan

Tabel 1.6 Kelas, Atribut, dan Method pada regularizers.py

Kelas	Deskripsi	Atribut	Method
L1Regularisasi	Mengimplementasikan penalti berdasarkan nilai absolut bobot. Cenderung menghasilkan bobot yang jarang (sparse)	• lambda_: Koefisien kekuatan regulerisasi λ	• loss(weights): Menghitung $\Sigma\lambda$
L2Regularizer	Mengimplementasikan penalti berdasarkan kuadrat nilai bobot	• lambda_: Koefisien kekuatan regulerisasi λ	• loss(weights): Menghitung $\lambda\Sigma\omega^2$

	(Ridge). Mendorong bobot agar bernilai kecil namun tidak nol.		• gradient(weights): Menghitung $2\lambda\omega$
NoRegularizer	Digunakan saat model tidak menerapkan teknik regulerisasi tambahan.	-	• loss(weights): Selalu mengembalikan 0.0 • gradient(weights): Mengembalikan matriks nol dengan shape yang sama dengan bobot
Helper Function			
Nama Fungsi	**Deskripsi**	**Parameter**	**Return**
get_regularizer	Fungsi pembantu untuk menginisialisasi objek regulerisasi berdasarkan nama string yang diberikan	• name (str): Jenis regulerisasi ("l1", "l2", atau "none") • **kwargs: Argumen tambahan untuk parameter lambda_	• Instance dari kelas regulerisasi yang dipilih.

b. Forward Propagation

Forward Propagation adalah proses ketika data (*input*) melewati setiap layer pada model yang diterapkan untuk menghasilkan output. Implementasi Forward Propagation dirancang sebagai proses aliran data sekuensial yang dimulai dari *input layer* hingga menghasilkan prediksi akhir pada *output layer*. Proses ini dikelola oleh FFNN *class* melalui forward(x) *method*, yang mengiterasi daftar objek DenseLayer yang telah disusun sesuai arsitektur jaringan. Setiap data *input* (X) yang diterima dalam bentuk *mini-batch* akan diteruskan dari satu *layer* ke *layer* berikutnya. Setiap lapisan bertanggung jawab untuk melakukan transformasi linear dan aktivasi non-linear.

Di dalam kelas DenseLayer, proses *forward propagation* melibatkan dua tahapan komputasi utama yang hasilnya disimpan dalam variabel *cache* untuk kebutuhan efisiensi memori pada tahap

backpropagation. Tahap pertama adalah perhitungan nilai pre-aktivasi (z), yang diperoleh melalui operasi perkalian matriks (dot product) antara data masukan dengan matriks bobot (W), kemudian ditambahkan dengan vektor bias (b). Nilai z ini disimpan ke dalam z_cache. Tahap kedua adalah penerapan fungsi aktivasi (seperti ReLU, Sigmoid, atau Softmax) pada nilai z untuk menghasilkan nilai aktivasi (a), yang kemudian disimpan ke dalam a_cache. Output aktivasi dari sebuah lapisan inilah yang kemudian menjadi input bagi *layer*berikutnya dalam jaringan.

Setelah data melintasi seluruh *layer*, hasil akhir dari *layer* terakhir akan dikembalikan sebagai prediksi model. Pada tahap ini, program juga menyediakan metode compute_loss untuk mengevaluasi kualitas prediksi tersebut dengan membandingkannya terhadap label target (y) menggunakan fungsi kerugian yang dipilih. Jika model menggunakan komponen regulerisasi (L1 atau L2), nilai pinalti dari bobot di setiap *layer* akan dihitung dan ditambahkan ke dalam total loss untuk mengontrol kompleksitas model selama pelatihan.

c. Backward Propagation dan Weight Update

Implementasi backward propagation dimulai dari perhitungan gradien *loss function* terhadap hasil prediksi model. Kelas-kelas *loss* seperti MSE, BinaryCrossEntropy, dan CategoricalCrossEntropy memiliki metode backward yang menghasilkan turunan awal, di mana nilainya dibagi dengan jumlah data n. Setelah nilai gradien awal diperoleh, model akan melakukan iterasi secara terbalik melalui daftar layer yang tersimpan. Gradien ini kemudian diteruskan mundur ke setiap layer sebagai parameter upstream_grad untuk diproses pada fungsi backward milik masing-masing *layer*.

Di dalam kelas DenseLayer, penerimaan nilai upstream_grad diproses untuk menghasilkan tiga komponen gradien matematis. Pertama, program menghitung gradien pre-aktivasi (dz), di mana jika *layer*menggunakan aktivasi Softmax, perhitungannya melibatkan manipulasi dimensi matriks Jacobian melalui operasi np.einsum. Untuk fungsi aktivasi selain Softmax, dz dihitung menggunakan perkalian *element-wise* antara upstream_grad dengan turunan fungsinya. Selanjutnya, turunan bobot (dW) dihitung melalui dot

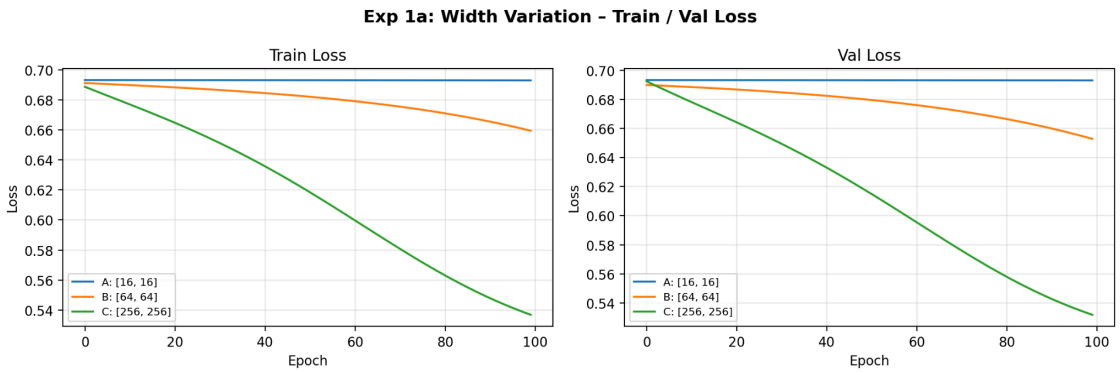
product antara transpos variabel input cache dengan dz yang dibagi dengan ukuran *batch*, lalu ditambahkan dengan gradien dari fungsi regularisasinya. Turunan bias (db) diperoleh dengan menjumlahkan elemen-elemen dz pada sumbu kolom yang juga dirata-rata berdasarkan ukuran batch. Terakhir, gradien input (dx) dihasilkan dari dot product antara dz dan transpos matriks bobot W, yang kemudian dikembalikan untuk menjadi nilai upstream_grad bagi layer sebelumnya.

Setelah penelusuran balik selesai melintasi seluruh layer, tahapan terakhir adalah pembaruan parameter jaringan menggunakan algoritme Gradient Descent. Melalui pemanggilan fungsi update_weights, model memperbarui jaringan dengan cara mengurangi nilai bobot W dan bias b pada setiap layer menggunakan hasil perkalian antara variabel learning rate dan masing-masing matriks gradien (dW dan db) yang telah dihitung.

2. Hasil Pengujian

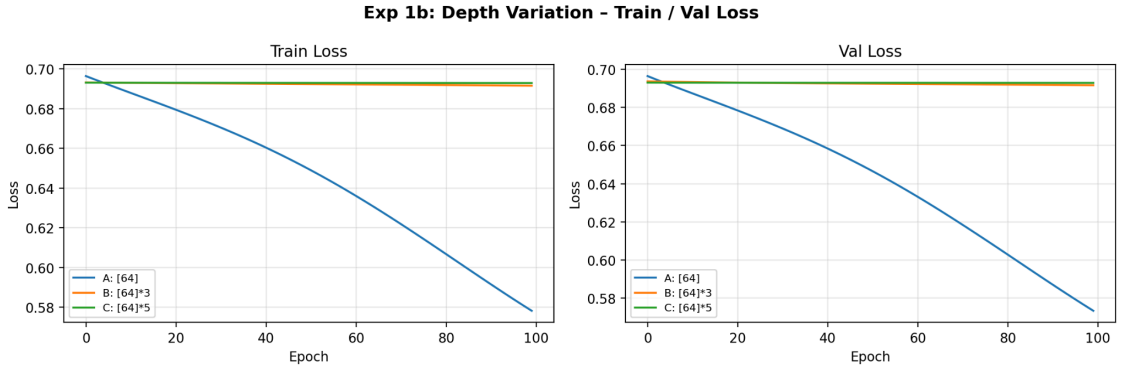
a. Pengaruh *depth* dan *width*

Config str	Accuracy float64	F1 float64	Precision float64	Recall float64
A: [16, 16]	0.573	0.5779	0.608	0.573
B: [64, 64]	0.745	0.7441	0.7435	0.745
C: [256, 256]	0.746	0.7477	0.7512	0.746



Semakin width bertambah, nilai metrik skor jaringan semakin tinggi. Model mempelajari lebih baik.

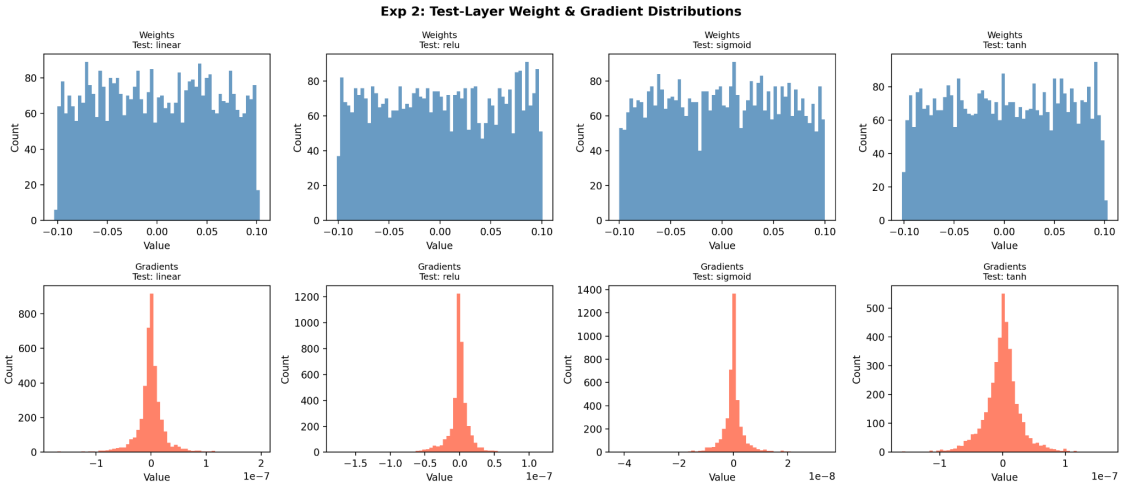
Config str	Accuracy float64	F1 float64	Precision float64	Recall float64
A: [64]	0.7435	0.745	0.7479	0.7435
B: [64]*3	0.6675	0.6717	0.6978	0.6675
C: [64]*5	0.661	0.6487	0.6502	0.661



Semakin depth bertambah, nilai metrik skor jaringan semakin kecil. Hal ini dapat terjadi karena model menjadi overfitting terhadap data.

b. Pengaruh fungsi aktivasi

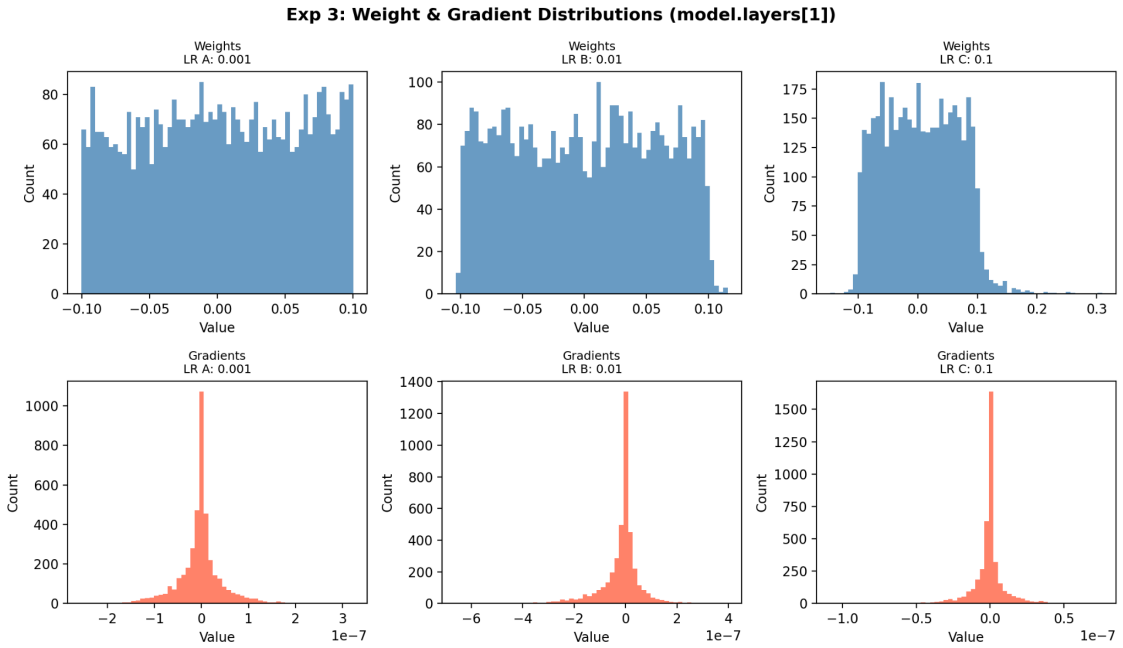
Activation str	Accuracy float64	F1 float64	Precision float64	Recall float64
linear	0.6865	0.6905	0.7118	0.6865
relu	0.672	0.6722	0.6723	0.672
sigmoid	0.5985	0.6029	0.6126	0.5985
tanh	0.7025	0.7051	0.711	0.7025



Pada eksperimen ini, fungsi aktivasi tanh hampir memiliki dominasi skor jika dibandingkan dengan fungsi-fungsi lain.

c. Pengaruh *learning rate*

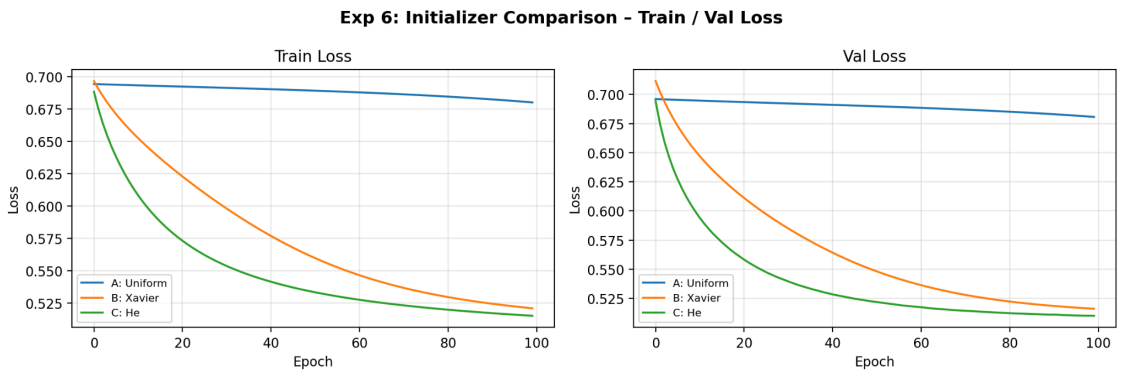
LR config str	Accuracy float64	F1 float64	Precision float64	Recall float64
LR A: 0.001	0.6395	0.644	0.6604	0.6395
LR B: 0.01	0.7455	0.7426	0.7422	0.7455
LR C: 0.1	0.753	0.755	0.7596	0.753



Learning rate yang lebih tinggi pada eksperimen ini menghasilkan skor yang lebih tinggi juga. Ada kemungkinan bahwa learning rate tinggi menyebabkan *convergence* yang lebih cepat.

d. Pengaruh inisialisasi bobot

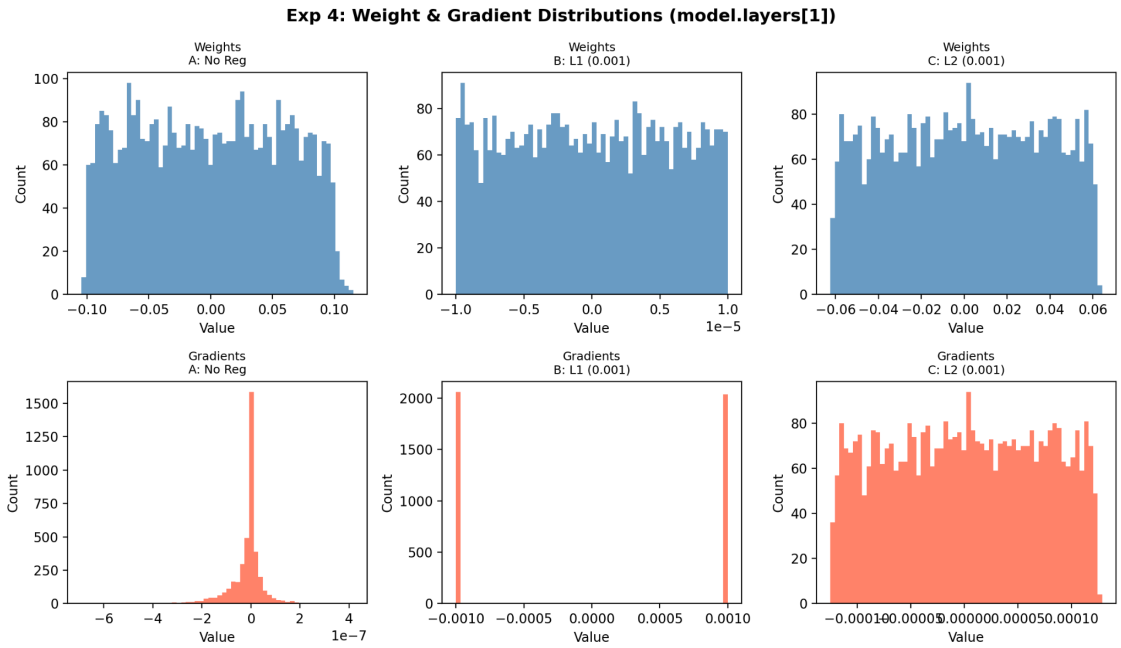
Config str	Accuracy float64	F1 float64	Precision float64	Recall float64
A: Uniform	0.704	0.7076	0.7221	0.704
B: Xavier	0.7485	0.75	0.7526	0.7485
C: He	0.742	0.7442	0.7493	0.742



Bobot yang diinisalisasi dengan initializer Xavier menghasilkan skor yang lebih tinggi dibandingkan dengan inisialisasi bobot lain. Namun, loss lebih cepat turun jika menggunakan initializer He.

e. Pengaruh regularisasi

Regularizer str	Accuracy float64	F1 float64	Precision float64	Recall float64
A: No Reg	0.737	0.7392	0.7446	0.737
B: L1 (0.001)	0.3875	0.2164	0.1502	0.3875
C: L2 (0.001)	0.705	0.7081	0.717	0.705



Tanpa regularisasi, weight nerkumpul pada satu tempat. Dengan L1, distribusi weight bernilai sparse. Dengan L2, distribusi weight lebih smooth.

f. Perbandingan dengan library sklearn

Model str	Accuracy float64	F1 float64	Precision float64	Recall float64
From scratch FFNN	0.6925	0.6963	0.7251	0.6925
sklearn MLP	0.704	0.7032	0.7025	0.704

Dalam beberapa metrik pengukuran, model kami kalah dibandingkan dengan implementasi oleh scikit-learn.

Kesimpulan dan Saran

1. Kesimpulan

Model Feedforward Neural Network (FFNN) berhasil diimplementasikan dari awal tanpa library eksternal dan mampu memprediksi prospek karier mahasiswa pada Global Student Placement Dataset. Performa jaringan sangat bergantung pada konfigurasi arsitektur. Jaringan yang lebih lebar (width), aktivasi tanh, inisialisasi Xavier, dan learning rate tinggi memberikan metrik skor terbaik. Sebaliknya, menambah kedalaman jaringan (depth) terbukti memicu terjadinya overfitting. Meskipun model yang dibuat secara mandiri sudah berjalan dengan baik dan sesuai konsep, performa akhirnya masih belum bisa mengungguli optimasi model Multi Layer Perceptron (MLP) bawaan dari library scikit-learn.

2. Saran

Berikut adalah beberapa saran yang dapat dipertimbangkan dalam pengembangan model FFNN dalam *project* ini untuk ke depannya.

- Mengingat penambahan depth menyebabkan overfitting, disarankan untuk mengimplementasikan teknik regulerisasi tambahan seperti Dropout atau Early Stopping pada pengembangan selanjutnya.
- Mengembangkan implementasi model dengan menambahkan algoritma *optimizer* yang lebih mutakhir (seperti Adam atau RMSprop) untuk membantu model mencapai konvergensi yang lebih optimal dan stabil dibandingkan *Gradient Descent* konvensional.
- Menggunakan pendekatan sistematis seperti Grid Search atau Random Search untuk mencari kombinasi keseluruhan hyperparameter yang paling optimal, alih-alih mengujinya satu per satu.

Pembagian Tugas

Nama/NIM	Pembagian Tugas
Wisa Ahmaduta DInutama/18223003	• Melakukan dataset preprocessing • Melakukan *experiments* • Melakukan pengujian • Mengerjakan laporan
Muhammad Faiz Alfikrona/18223009	• Mengerjakan regularizers.py • Mengerjakan layers.py • Mengerjakan model.py • Melakukan pengujian • Mengerjakan laporan
Ni Made Sekar Jelita/18223101	• Mengerjakan activations.py • Mengerjakan initializers.py • Mengerjakan losses.py • Mengerjakan laporan

Referensi

Backpropagation in neural network. (2024, March 4). GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/backpropagation-in-neural-network/

Feedforward neural network. (2024, June 20). GeeksforGeeks. https://www.geeksforgeeks.org/deep-learning/feedforward-neural-network/

Feedforward neural network. (2025). In *Wikipedia*. https://en.wikipedia.org/w/index.php?title=Feedforward_neural_network&oldid=1326004759

What is forward propagation in neural networks. (2024, May 1). GeeksforGeeks. https://www.geeksforgeeks.org/deep-learning/what-is-forward-propagation-in-neural-networks/

SURAT PERNYATAAN PENGGUNAAN AI

Nama/NIM	:	Wisa Ahmaduta Dlnutama/18223003
	:	Muhammad Faiz Alfikrona/18223009
	:	Ni Made Sekar Jelita/18223101
Fakultas/Sekolah	:	Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah	:	IF3270
Nama Mata Kuliah	:	Pembelajaran Mesin
Judul Tugas/Proposal	:	Tugas Besar 1 FeedForward Neural Network

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	X	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	V	Sedang
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	V	Tinggi
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	X	-

Bandung, 18 Maret 2026

Wisa Ahmaduta DInutama	Muhammad Faiz Alfikrona	Ni Made Sekar Jelita
18223003	18223009	18223101